

agent-meow 实施范围报告

计划 001-010 · 完整实施路线图

运行环境： Colorfire Meow Series R16 395 AI 笔记本

芯片： AMD Ryzen AI MAX+ 395 (Strix Halo) · CPU + NPU + iGPU 全栈优化

报告日期： 2026-08-04 · **状态：** 全部已推送至 `origin/main`

硬件平台 — Colorfire R16 395 AI

组件	规格	状态
CPU	AMD Ryzen AI MAX+ 395, 16C/32T, 3.0GHz	✓
iGPU	AMD Radeon 8060S, Vulkan 1.4.329	✓
iGPU 显存	96GB (统一内存, 驱动确认)	✓
NPU	AMD XDNA 2 (PCI 已识别)	✓
内存	32GB DDR5	✓
ROCm	7.1 + HIP 7.1.51803	✓
Ollama	0.32.5, meow36b + qwen3.6:35b-a3b (38GB)	✓
品牌	COLORFIRE LinC K16 (喵星系列)	BIOS 确认

关键技术声明：iGPU 显存由 AMD 统一内存架构提供（`qwMemorySize=103079215104` 字节 $\approx$ 96GB）。Windows 32 位显示驱动仅报告 4GB，但 ROCm/HIP 运行时可见全部 96GB。这使 38GB 的 LLM 模型可完全驻留 GPU 显存，无需 CPU↔GPU 交换。

为什么 Strix Halo 让 agent-meow 独特

普通 PC 架构：CPU 计算 + GPU 图形 + 无 NPU，各芯片独立内存池

Strix Halo 架构：CPU + iGPU(96GB) + NPU 三引擎并行，**统一内存池**，零拷贝共享

Colorfire R16 · Strix Halo · iGPU 96GB 统一显存

CPU (16核)

- └ Kokoro TTS (82M)
- └ VAD Silero
- └ QAA Node.js 网关
- └ Vite 前端开发服务器

iGPU (Radeon 8060S)

- └ Ollama qwen3.6:35b-a3b LLM (38GB)
- └ whisper.cpp STT (Vulkan 后端)

NPU (XDNA 2)

- └ 未来：NPU STT (winml, 待 2026 末)

设计原则：每个推理引擎分配到最适合的硬件。LLM 和 STT 在 iGPU（大显存、高吞吐），TTS 和 VAD 在 CPU（小模型、低延迟启动），NPU 保留给未来低功耗 STT。统一内存意味着 GPU 模型加载无需 PCIe 拷贝——38GB 模型在 ~3 秒内就绪。

计划 001–005：基础设施修复

#	计划	状态	内容
001	修复 db_models 路径	TODO	路径精确化
002	同步 VIDEOS_SURFACE.md	TODO	2 项过时声明
003	Phase 4 runner 调度	TODO	注册工具到 runner
004	标记过时 voicebox 计划	TODO	清理废弃计划
005	Voicebox 引擎可靠性	DRAFT	预 QAA 方案

优先级原则：001–005 是代码卫生修复，不改变用户可观测行为。它们优先级低于 QAA 语音迁移（006–010），因为语音是用户感知最强的痛点（90 秒预热）。005 已被 006+008 方案取代——原计划用 faster-whisper 优化，但根因分析发现 CUDA-only 限制无法在 AMD GPU 上绕过。

计划 006：QAA 网关 + DashScope 云端

在线语音模式 · 解决 90 秒预热痛点

问题陈述：当前 S2S 语音栈使用 faster-whisper (STT) + Kokoro (TTS)，冷启动 90 秒。根因是 faster-whisper 基于 CTranslate2，**仅支持 CUDA**，AMD Radeon 8060S 的 96GB 显存完全闲置，STT 在 CPU 上跑 60 秒。

解决方案：安装 Qwen-Audio-Agent (QAA) v1.3.0 作为语音网关，配置 DashScope `qwen-audio-3.0-realtime-flash` 作为在线 S2S provider。DashScope 是阿里云实时语音服务，原生支持 OpenAI Realtime API 协议，中文质量优秀，中国大陆可直连。

指标	当前	优化后
预热	90 秒	~0 秒（云端常驻）
中文	✓	✓（原生）
成本	免费	90 天免费, 后 ~¥0.20/分
网络	无	需要互联网

实施步骤：DashScope API 密钥 → `npm install QAA` → 配置 dual-provider → 冒烟测试

风险: LOW · **工作量:** S · **依赖:** 无

计划 006b：在线/离线混合部署接线

浏览器 → QAA → DashScope/S2S 完整数据通路

QAA 原生支持**每会话 provider 切换**——两个 provider 同时配置，运行时选择。这使混合模式成为架构级能力，而非 hack。

```
浏览器 (:5173)
  → Vite 代理 (/api→:3101, ws:true)
    → QAA 网关 (:3101, Node.js)
      ├── ☁ 在线 → DashScope 云端 (~0s, qwen-audio-3.0-realtime-flash)
      ├── 🏠 离线 → 本地 S2S (:8765, whisper.cpp+Kokoro)
      └── 工具 → Hermes/Ollama 后端 (ACP 协议)
```

关键技术点：Vite 配置必须设 `ws: true` 以启用 WebSocket 代理（实时语音用 WS 非 HTTP）。自动回退逻辑：DashScope 连接失败或超时时，网关透明切换到离线 S2S，用户无感知。这解决了中国网络环境下的可用性问題——即使云端不可达，语音仍可工作。

计划 007：移植 QAA 语音钩子到 MeowCat 界面

保留猫爪 UI，替换底层传输协议

设计原则：UI 层（猫爪按钮、波形动画）是 agent-meow 的品牌标识，不变。传输层（WebSocket 协议、事件格式）从手写实现替换为 QAA 的成熟实现。

旧组件	新组件	动作
realtimeVoice.ts (221行)	QAA useRealtimeVoice.js	替换
s2s_proxy.py (233行)	QAA 网关	替换
猫爪按钮+波形	保留	不变
在线/离线切换	新增	☁️/🏠

协议差异：QAA 使用 GatewayClientEvent JSON 协议（type + audio_delta / audio_commit），当前手写实现使用自定义二进制帧。需重写事件处理器，但 MeowCat 的 React 组件树不变。风险在于协议映射完整性——所有音频帧、中断、VAD 事件必须正确映射。**风险:** MED · **工作量:** L

计划 008: whisper.cpp + Vulkan GPU 加速 STT

离线预热优化 · GPU 从闲置到全速

根因分析：faster-whisper 基于 CTranslate2，**仅支持 CUDA**。AMD Radeon 8060S 有 96GB 显存但完全闲置——STT 在 CPU 上跑 60 秒。这是 90 秒预热的主要来源。

技术方案：whisper.cpp v1.9.1 支持 Vulkan 后端 (GGML_VULKAN=1) 和 HIP 后端 (GGML_HIP=1)。选择 Vulkan 因为其跨厂商兼容性更好，且 Windows 上 ROCm/HIP 支持仍不稳定。将 STT 推理放到 iGPU。

组件	当前	优化后	位置
STT	CPU 60s	GPU ~3s	iGPU
TTS	CPU 30s	~0s (预热)	CPU
总预热	90s	~8s 或 0s	—

显存预算：96GB iGPU 显存同时容纳 whisper-large-v3 (~1.5GB) + Ollama LLM (38GB) = 39.5GB，剩余 56.5GB 充裕。TTS 改为启动时预热 Kokoro-82M，消除 30 秒冷启动。**风险:** MED · **工作量:** M

计划 009: ACP 垫片 → Hermes 代理 OS

非阻塞工具调用语音 (Path B)

问题：标准 S2S 模式下，AI 在"思考"时用户无法说话（半双工）。工具调用（代码执行、文件操作）会阻塞语音通道数十秒，用户体验差。

解决方案：ACP (Agent Communication Protocol) 垫片将语音前端与 Hermes 代理 OS 解耦。简单问题由 QAA 实时前端即时回答；需要工具调用时，`spawn_thinking` 在后台异步执行，完成后语音播报结果。

用户说话 → QAA 实时前端 → 简单问题? → 即时语音回答
→ 需要工具? → `spawn_thinking` → ACP 垫片 → Hermes
→ 代码/文件/MCP 工具调用 → 完成后语音播报

Hermes = 代理 OS：集成 skills + memory + cron + terminal + MCP。ACP 垫片让语音成为 Hermes 的一等交互界面，而非附属功能。**风险:** MED · **工作量:** L

计划 010：本地 Ollama LLM · Strix Halo 全栈

从 Hermes Docker 到完全本地 GPU 推理

动机：当前 Hermes 跑在 Docker 容器内，LLM 推理通过远程 API 调用（延迟 ~1.8ms localhost RTT，但仍有序列化开销）。Strix Halo 的 96GB iGPU 显存可以完全容纳 38GB 的 qwen3.6:35b-a3b 模型，实现零延迟本地推理。

指标	Hermes Docker	Ollama 本地
内存	~1-2GB (VM)	~0 (原生)
启动	~5-10s	~3s
推理	远程 API	GPU (ROCm)
延迟	~1.8ms	~0.3ms
成本	API 费用	零

技术基础：Ollama 0.32.5 已安装，meow36b + qwen3.6:35b-a3b-q8_0 (38GB) 已就绪。ROCm 7.1 提供 HIP 后端，Ollama 原生支持 `HIP_VISIBLE_DEVICES` 选 GPU。38GB 模型在 96GB 显存中占 40%，无交换。**风险：**MED · **工作量：**M

完整 Strix Halo 优化栈

CPU + iGPU + NPU 三引擎分工

组件	引擎	位置	预热	计划
LLM (qwen3.6:35b)	Ollama+ROCm	iGPU 96GB	~3-5s	010
STT (Whisper)	whisper.cpp+Vulkan	iGPU	~3s	008
TTS (Kokoro)	Kokoro-82M	CPU	~0s	008
VAD (Silero)	Silero	CPU	~0s	现有
语音网关	QAA (Node.js)	CPU	~2s	006
代理 OS	Hermes/Ollama	CPU+iGPU	已运行	009/010
前端	React+Vite	浏览器	即时	007
NPU STT	未来 (winml)	NPU	待 2026 末	未来

显存预算： iGPU 96GB = LLM 38GB + STT 1.5GB = 39.5GB，剩余 56.5GB 充裕。统一内存架构意味着 CPU 和 GPU 共享同一物理内存池，无 PCIe 拷贝开销。NPU 路径（winml CLI，ONNX 量化模型）留待 2026 末 AMD 驱动成熟后实施。

依赖关系与执行顺序

006 (QAA+DashScope) → 007 (移植语音钩子)
006 → 009 (ACP 垫片)
008 (whisper.cpp) → 007 (并行, 独立)
009 → 010 (Ollama LLM)
006b (混合接线) → 007

阶段	计划	说明
1	006 + 008	并行, 独立 (无相互依赖)
2	006b + 007	依赖 006 (需 QAA 网关就绪)
3	009	依赖 006 (需 ACP 协议端点)
4	010	依赖 009 (需 Hermes→Ollama 桥接)

调度原则：006 和 008 可完全并行——前者安装云端网关，后者构建本地 GPU STT，无冲突。007 必须在 006 和 008 都完成后进行，因为它同时依赖 QAA 协议和离线 STT 后端。

预期最终效果

指标	当前	最终目标
语音预热	90s	~0s 在线 / ~8s 离线
LLM 推理	远程 API	本地 GPU (ROCm, 96GB)
STT 推理	CPU 60s	GPU Vulkan ~3s
TTS 预热	30s	~0s (预热)
云端依赖	必须	可选 (混合)
成本	API 费用	零 (离线)
GPU 利用率	0%	STT + LLM 在 GPU
NPU 利用率	0%	未来 STT
Docker	必须	可选 (Ollama)
代理能力	无	Hermes OS (代码/文件/MCP)

最终愿景： agent-meow 在 Colorfire R16 395 AI 上成为唯一为 Strix Halo 三引擎（CPU+iGPU+NPU）全面优化的本地 AI 语音代理。LLM 在 GPU（Ollama+ROCm），STT 在 GPU（whisper.cpp+Vulkan），TTS 在 CPU（Kokoro），代理 OS 在后台（Hermes），NPU 留给未来。全部共享 96GB 统一显存，零云端，零外部 GPU。